

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CƠ BẢN I

BỘ MÔN ĐỒ ÁN THIẾT KẾ MẠCH ĐIỆN TỬ



Đồ Án Thiết Kế Mạch Điện Tử
Đề tài: Thiết kế chế tạo thử nghiệm hệ thống IoT
giám sát môi trường và điều khiển thiết bị thời gian
thực qua Internet

Giảng viên hướng dẫn	: TRƯƠNG MINH ĐỨC
Họ và tên sinh viên	: NGUYỄN QUANG LÂM
Mã sinh viên	: B22DCDT176
Lớp	: Đồ án TKMĐT
Nhóm	: 03

Lời Cảm Ơn

Em xin gửi lời cảm ơn chân thành đến **thầy Trương Minh Đức**, giảng viên bộ môn, người đã tận tình hướng dẫn, hỗ trợ và định hướng em trong suốt quá trình thực hiện đồ án.

Nhờ những ý kiến đóng góp quý báu cùng sự đồng hành của thầy, em đã có thể hoàn thành đề tài: **"Thiết kế hệ thống giám sát môi trường và điều khiển thiết bị sử dụng ESP32"** một cách hiệu quả và có định hướng rõ ràng.

Em cũng xin trân trọng cảm ơn các thầy cô trong khoa đã truyền đạt cho em những kiến thức nền tảng và kinh nghiệm thực tiễn trong suốt quá trình học tập tại trường.

Mặc dù đã cố gắng hoàn thành tốt nhất trong khả năng, nhưng do hạn chế về thời gian và kinh nghiệm, báo cáo không thể tránh khỏi những thiếu sót. Em mong nhận được sự góp ý, đánh giá từ thầy cô để em có thể rút kinh nghiệm và hoàn thiện hơn trong những lần sau.

Em xin chân thành cảm ơn!

MỤC LỤC

1. Tóm tắt nội dung báo cáo

1.1: Tóm lược về đề tài:

Trong những năm gần đây, sự phát triển nhanh chóng của công nghệ điện tử và IoT đã mở ra nhiều cơ hội cho các mô hình sản xuất nhỏ lẻ tiếp cận các giải pháp giám sát thông minh. **Nếu như trước kia, các hệ thống giám sát môi trường chỉ được lựa chọn bởi những cơ sở chăn nuôi, trồng trọt quy mô lớn do chi phí đầu tư cao và yêu cầu kỹ thuật phức tạp, thì nay, chúng đã bắt đầu xuất hiện tại cả những mô hình chuồng trại nhỏ và vừa** – nhờ vào sự ra đời của các vi điều khiển nhỏ gọn, cảm biến giá rẻ và khả năng kết nối không dây dễ dàng.

Từ thực tế đó, đề tài này được thực hiện nhằm phát triển một hệ thống giám sát môi trường tích hợp, sử dụng vi điều khiển ESP32 cùng các cảm biến phổ biến, với tiêu chí **chi phí thấp – dễ triển khai – có khả năng mở rộng**, phù hợp với hộ gia đình, trang trại quy mô nhỏ hoặc các cơ sở sản xuất vừa và nhỏ trong thực tế.

1.2: Tóm lược về các thành phần phần cứng:

Để xây dựng hệ thống giám sát môi trường và điều khiển thiết bị, đề tài sử dụng các thiết bị phần cứng chính sau:

1. Vi điều khiển ESP32

Là bộ xử lý trung tâm của hệ thống, ESP32 có khả năng kết nối WiFi mạnh mẽ, hiệu năng cao và tích hợp nhiều tính năng cần thiết để thu thập, xử lý và truyền dữ liệu từ các cảm biến đến thiết bị hiển thị hoặc nền tảng đám mây.

2. Cảm biến BH1750

Dùng để đo cường độ ánh sáng môi trường (lux). BH1750 có độ chính xác cao, giao tiếp qua chuẩn I2C, giúp hệ thống theo dõi được điều kiện ánh sáng trong khu vực giám sát.

3. Cảm biến SHT31

Đảm nhiệm việc đo nhiệt độ và độ ẩm không khí với độ chính xác tốt. SHT31 cũng giao tiếp qua I2C, giúp việc tích hợp với ESP32 trở nên đơn giản và tiện lợi.

4. Màn hình OLED 1.3 inch (điện áp 3.3V)

Dùng để hiển thị thông số đo được theo thời gian thực như nhiệt độ, độ ẩm, cường độ ánh sáng,... giúp người dùng dễ dàng quan sát trực tiếp tại thiết bị mà không cần thông qua thiết bị thứ cấp.

5. Servo SG90

Đóng vai trò thiết bị điều khiển mô phỏng, có thể dùng để vận hành các thiết bị như cửa thông gió, van nước hoặc các cơ cấu điều khiển cơ học khác theo tín hiệu từ ESP32.

1.3: Hướng giải quyết:

Để giải quyết bài toán giám sát môi trường một cách hiệu quả, đề tài lựa chọn sử dụng vi điều khiển ESP32 làm bộ xử lý trung tâm nhờ khả năng kết nối WiFi và hiệu năng cao. Các cảm biến BH1750 và SHT31 được tích hợp để đo chính xác các thông số môi trường cơ bản như cường độ ánh sáng, nhiệt độ và độ ẩm. Dữ liệu thu thập được hiển thị trực tiếp trên màn hình OLED 1.3 inch giúp người dùng theo dõi dễ dàng tại chỗ.

Ngoài ra, việc sử dụng servo SG90 cho phép mô phỏng điều khiển thiết bị cơ khí như cửa thông gió hoặc van, giúp hệ thống không chỉ giám sát mà còn có thể phản hồi tự động theo điều kiện môi trường.

Toàn bộ hệ thống được thiết kế theo hướng **tối giản, tiết kiệm chi phí và dễ dàng mở rộng**, phù hợp với hộ gia đình, trang trại và các đơn vị sản xuất quy mô nhỏ. Việc kết nối WiFi cho phép mở rộng chức năng giám sát và điều khiển từ xa qua điện thoại hoặc máy tính.

1.4: Kết quả mong muốn:

- Hệ thống hoạt động ổn định, có khả năng đo chính xác nhiệt độ, độ ẩm và cường độ ánh sáng trong môi trường thực tế.
- Dữ liệu được hiển thị trực tiếp trên màn hình OLED giúp quan sát dễ dàng.
- Servo SG90 phản hồi nhanh, có thể điều khiển các thiết bị cơ khí theo yêu cầu.
- Thiết bị nhỏ gọn, chi phí thấp và dễ dàng lắp đặt trong các mô hình chuồng trại hoặc nhà xưởng nhỏ.
- Hệ thống có tiềm năng mở rộng để tích hợp điều khiển từ xa qua WiFi hoặc nền tảng đám mây trong các phiên bản nâng cấp sau.

2. Giới thiệu đề tài:

2.1: Lý do chọn đề tài:

Trong bối cảnh phát triển nhanh chóng của công nghệ IoT và sự gia tăng nhu cầu giám sát môi trường trong sản xuất nông nghiệp, chăn nuôi và các ngành công nghiệp vừa và nhỏ, việc thiết kế một hệ thống giám sát môi trường thông minh, hiệu quả và tiết kiệm chi phí trở nên rất cần thiết.

Các hệ thống giám sát truyền thống thường có chi phí cao, quy mô lớn, khó tiếp cận đối với các hộ gia đình hoặc trang trại nhỏ. Đồng thời, việc theo dõi các thông số môi trường như nhiệt độ, độ ẩm và ánh sáng một cách liên tục giúp nâng cao hiệu quả quản lý, bảo vệ sức khỏe vật nuôi, cây trồng và tối ưu hóa quy trình sản xuất.

Do đó, đề tài “Thiết kế hệ thống giám sát môi trường và điều khiển thiết bị sử dụng ESP32” được lựa chọn nhằm phát triển một giải pháp công nghệ đơn giản, tiết kiệm và dễ áp dụng, phù hợp với nhu cầu thực tế của các mô hình sản xuất nhỏ và vừa, góp phần thúc đẩy chuyển đổi số trong nông nghiệp và công nghiệp hiện đại.

2.2: Mục tiêu đề tài:

Mục tiêu chính của đề tài là thiết kế và triển khai một hệ thống giám sát môi trường thông minh, tích hợp các cảm biến đo nhiệt độ, độ ẩm và cường độ ánh sáng, đồng thời có khả năng điều khiển thiết bị cơ khí đơn giản (như servo SG90) để phản hồi tự động theo điều kiện môi trường.

Cụ thể, đề tài hướng đến các mục tiêu sau:

- Xây dựng hệ thống sử dụng vi điều khiển ESP32 với khả năng kết nối WiFi, đảm bảo truyền tải dữ liệu hiệu quả và ổn định.
- Tích hợp cảm biến BH1750 và SHT31 để đo chính xác các thông số môi trường quan trọng như ánh sáng, nhiệt độ và độ ẩm.
- Hiển thị dữ liệu đo được trên màn hình OLED 1.3 inch để người dùng có thể dễ dàng quan sát trực tiếp.
- Sử dụng servo SG90 để điều khiển các thiết bị cơ khí nhỏ theo tín hiệu từ hệ thống, hỗ trợ tự động hóa trong quản lý môi trường.
- Thiết kế hệ thống có chi phí thấp, cấu hình đơn giản, dễ lắp đặt và vận hành, phù hợp với các hộ gia đình, trang trại và doanh nghiệp nhỏ.
- Mở rộng khả năng giám sát và điều khiển từ xa qua kết nối WiFi trong các phiên bản nâng cao sau này.

2.3: Phạm vi và giới hạn đề tài:

Đề tài tập trung thiết kế một hệ thống giám sát môi trường tích hợp các cảm biến đo nhiệt độ, độ ẩm và cường độ ánh sáng, sử dụng vi điều khiển ESP32 làm trung tâm xử lý. Hệ thống có khả năng hiển thị dữ liệu trực tiếp trên màn hình OLED 1.3 inch và điều khiển servo SG90 để mô phỏng các thiết bị cơ khí đơn giản.

Phạm vi đề tài giới hạn ở việc thu thập và xử lý dữ liệu môi trường trong các mô hình nhỏ và vừa như trang trại, nhà kính hoặc nhà xưởng quy mô nhỏ. Đề tài không bao gồm phát triển phần mềm điều khiển phức tạp trên điện thoại hay máy tính, cũng không tích hợp các cảm biến khác ngoài BH1750 và SHT31. Ngoài ra, hệ thống chưa áp dụng các thuật toán phân tích dữ liệu nâng cao hay học máy.

2.4: Ý nghĩa và ứng dụng thực tiễn:

Hệ thống giám sát môi trường được thiết kế trong đề tài mang lại giải pháp hiệu quả, tiết kiệm và dễ dàng triển khai cho các hộ gia đình, trang trại nhỏ và doanh nghiệp vừa và nhỏ. Việc theo dõi liên tục các thông số môi trường giúp nâng cao hiệu quả quản lý, bảo vệ sức khỏe vật nuôi và cây trồng, từ đó tăng năng suất và giảm thiểu rủi ro trong sản xuất.

Bên cạnh đó, việc ứng dụng vi điều khiển ESP32 kết hợp với các cảm biến phổ biến và mô-đun điều khiển servo tạo nền tảng mở cho các hệ thống tự động hóa và giám sát thông minh trong tương lai, góp phần thúc đẩy quá trình chuyển đổi số trong nông nghiệp và công nghiệp quy mô nhỏ.

3. Cơ sở lý thuyết:

3.1: Các linh kiện được sử dụng:

3.1.1: Vi điều khiển ESP32:

ESP32 là một vi điều khiển (microcontroller) mạnh mẽ do công ty Espressif Systems phát triển, được sử dụng rộng rãi trong các ứng dụng Internet vạn vật (IoT) nhờ tích hợp sẵn các module WiFi và Bluetooth.

Đặc điểm chính:

- **Lõi xử lý:** ESP32 có lõi kép Tensilica Xtensa 32-bit, tốc độ xử lý lên tới 240 MHz, cho phép xử lý các tác vụ phức tạp nhanh và hiệu quả.
- **Bộ nhớ:** Tích hợp RAM 520 KB và bộ nhớ flash ngoài hỗ trợ lưu trữ chương trình, dữ liệu.
- **Kết nối không dây:** Hỗ trợ WiFi 802.11 b/g/n và Bluetooth v4.2 (Classic và BLE), thuận tiện cho việc truyền dữ liệu không dây.
- **Giao tiếp ngoại vi:** Cung cấp nhiều giao thức giao tiếp như I2C, SPI, UART, ADC, DAC, PWM,... giúp kết nối dễ dàng với các cảm biến và thiết bị ngoại vi.
- **Nguồn điện:** Hoạt động trong khoảng điện áp từ 2.2V đến 3.6V, tiêu thụ điện năng thấp, phù hợp với các thiết bị chạy pin.
- **Ứng dụng:** ESP32 phù hợp để xây dựng các hệ thống giám sát, điều khiển từ xa, thu thập dữ liệu cảm biến và các ứng dụng IoT đa dạng.



Tại sao chọn ESP32 cho đề tài?

- ESP32 có khả năng xử lý mạnh mẽ và tích hợp WiFi, rất phù hợp cho việc truyền dữ liệu môi trường lên các thiết bị khác hoặc nền tảng đám mây.
- Hỗ trợ nhiều giao tiếp ngoại vi giúp kết nối dễ dàng với cảm biến BH1750, SHT31 và màn hình OLED.

- Giá thành hợp lý, dễ dàng lập trình và có cộng đồng hỗ trợ rộng lớn.
- Tiêu thụ điện năng thấp, thích hợp cho các ứng dụng chạy liên tục trong môi trường thực tế.

Lý do không chọn STM32 và ESP8266

- STM32 không tích hợp WiFi/Bluetooth, cần thêm module ngoài, tăng chi phí và độ phức tạp.
- ESP8266 yếu hơn về hiệu năng, chỉ có WiFi, không hỗ trợ Bluetooth và ít giao tiếp ngoại vi hơn ESP32.

3.1.2: Cảm biến ánh sáng BH1750:

BH1750 là cảm biến ánh sáng kỹ thuật số, được sử dụng phổ biến trong các ứng dụng đo cường độ ánh sáng môi trường.

Đặc điểm chính:

- **Đo cường độ ánh sáng:** BH1750 đo ánh sáng với đơn vị lux, cho giá trị trực tiếp, dễ xử lý.
- **Giao tiếp:** Sử dụng chuẩn giao tiếp I2C, dễ dàng kết nối với vi điều khiển như ESP32.
- **Độ chính xác cao:** Có độ nhạy và độ chính xác tốt hơn so với cảm biến ánh sáng analog truyền thống.
- **Phản hồi nhanh:** Cập nhật giá trị ánh sáng liên tục, phù hợp giám sát môi trường thời gian thực.
- **Kích thước nhỏ gọn, tiêu thụ điện năng thấp,** phù hợp cho thiết bị di động hoặc chạy pin.



Tại sao chọn BH1750 cho đề tài?

- BH1750 cung cấp dữ liệu ánh sáng kỹ thuật số, giúp giảm sai số do nhiễu tín hiệu so với cảm biến analog.
- Giao tiếp I2C đơn giản, dễ tích hợp với ESP32 và các cảm biến khác.
- Độ chính xác và tốc độ phản hồi cao, phù hợp giám sát ánh sáng trong môi trường trang trại, nhà kính.
- Giá thành hợp lý và phổ biến trên thị trường.

Lý do không chọn cảm biến LDR

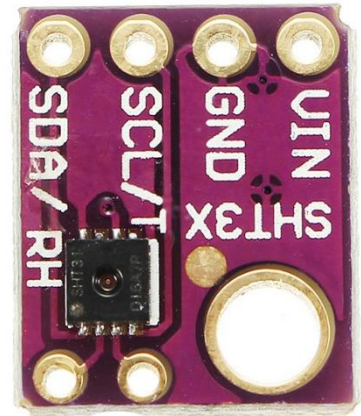
- LDR là cảm biến analog, dễ bị ảnh hưởng bởi nhiễu và không ổn định.
- Độ chính xác và khả năng tái lập kém hơn so với BH1750.
- Cần mạch chuyển đổi analog sang số phức tạp hơn, gây tăng chi phí và khó tích hợp.

3.1.3: Cảm biến nhiệt độ và độ ẩm SHT31:

SHT31 là cảm biến kỹ thuật số đo đồng thời nhiệt độ và độ ẩm, được sử dụng rộng rãi trong các ứng dụng giám sát môi trường.

Đặc điểm chính:

- **Đo nhiệt độ và độ ẩm:** Cung cấp dữ liệu chính xác và ổn định dưới dạng kỹ thuật số.
- **Giao tiếp:** Sử dụng giao thức I2C, dễ dàng kết nối với vi điều khiển như ESP32.
- **Độ chính xác cao:** Nhiệt độ sai số $\pm 0.3^{\circ}\text{C}$, độ ẩm sai số $\pm 2\% \text{ RH}$, vượt trội hơn so với nhiều cảm biến phổ biến.
- **Phạm vi đo:**
 - Nhiệt độ: từ -40°C đến $+125^{\circ}\text{C}$
 - Độ ẩm: từ 0% đến 100% RH
- **Phản hồi nhanh:** Cập nhật dữ liệu liên tục, phù hợp cho giám sát môi trường thực tế.
- **Tiêu thụ điện năng thấp,** kích thước nhỏ gọn, dễ tích hợp trong hệ thống.



Tại sao chọn SHT31 cho đề tài?

- Cung cấp dữ liệu nhiệt độ và độ ẩm chính xác, ổn định, phù hợp cho các ứng dụng giám sát môi trường trang trại, nhà kính.
- Giao tiếp I2C đơn giản, dễ dàng kết nối với ESP32 và các thiết bị ngoại vi khác.
- Độ bền cao và hoạt động tốt trong nhiều điều kiện môi trường khác nhau.
- Giá thành hợp lý, dễ mua trên thị trường.

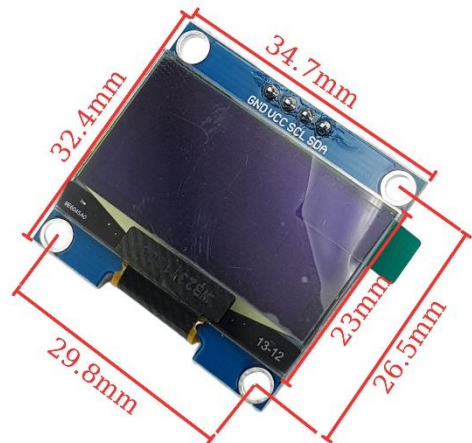
Lý do không chọn cảm biến DHT (DHT11, DHT22)

- Phạm vi đo hẹp hơn, ví dụ DHT11 chỉ đo được nhiệt độ từ 0°C đến 50°C và độ ẩm từ 20% đến 80%.
- Độ chính xác và độ ổn định thấp hơn, kết quả dễ bị nhiễu và sai số cao hơn.
- Tốc độ truyền dữ liệu chậm, không phù hợp cho giám sát liên tục và thời gian thực.
- Độ bền thấp, dễ bị ảnh hưởng bởi điều kiện môi trường khắc nghiệt.

3.1.4:Màn hình OLED 1.3 inch (I2C):

Đặc điểm chính

- Kích thước 1.3 inch, độ phân giải cao, hiển thị sắc nét, rõ ràng.
- Sử dụng công nghệ OLED nên cho độ tương phản cao, màu đen sâu và góc nhìn rộng.
- Giao tiếp qua chuẩn I2C, tiết kiệm chân kết nối và dễ dàng tích hợp với ESP32.
- Tiêu thụ điện năng thấp, thích hợp cho các thiết bị giám sát chạy liên tục.
- Kích thước nhỏ gọn, phù hợp cho các ứng dụng di động hoặc không gian hạn chế.



Lý do chọn màn hình OLED 1.3 inch

- Hiển thị sắc nét, rõ ràng trong nhiều điều kiện ánh sáng, đặc biệt là môi trường tối.
- Giao tiếp I2C giúp đơn giản hóa thiết kế phần cứng và giảm số chân kết nối.
- Tiêu thụ điện năng thấp, tăng thời gian hoạt động cho thiết bị.
- Kích thước nhỏ gọn, phù hợp với thiết bị giám sát môi trường.

Lý do không chọn màn hình LCD

- LCD thường tiêu thụ điện năng cao hơn so với OLED, không phù hợp cho thiết bị hoạt động liên tục với nguồn hạn chế.
- Độ tương phản và góc nhìn kém hơn, đặc biệt khi sử dụng trong môi trường ánh sáng yếu.
- Cần đèn nền (backlight) làm tăng tiêu thụ điện năng và giảm độ bền của màn hình.
- Kích thước thường lớn hơn và khó tích hợp trong các thiết bị nhỏ gọn.

3.1.5: Servo SG90:

Đặc điểm chính

- Là loại servo nhỏ gọn, phổ biến trong các dự án điều khiển góc quay.
- Góc quay tối đa khoảng 180 độ, đáp ứng tốt các ứng dụng điều khiển cơ bản.
- Điện áp hoạt động phổ biến: 4.8V – 6V.
- Dòng tiêu thụ thấp, thích hợp sử dụng với nguồn 5V ngoài hoặc từ mạch điều khiển.
- Điều khiển bằng tín hiệu PWM, dễ dàng kết nối và điều khiển bằng ESP32.
- Giá thành rẻ, dễ tìm mua trên thị trường.



Lý do chọn Servo SG90

- Kích thước nhỏ gọn, phù hợp với thiết kế tổng thể của hệ thống giám sát.
- Dễ dàng lập trình và điều khiển chính xác vị trí góc quay.
- Giá thành hợp lý, phù hợp cho dự án nghiên cứu và ứng dụng vừa và nhỏ.
- Tương thích tốt với nguồn 5V phổ biến và vi điều khiển ESP32.

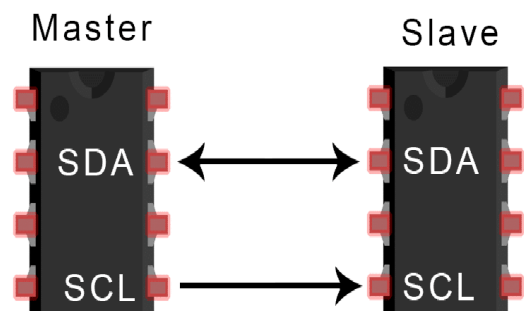
3.1.6: Giao thức I2C trong hệ thống:

Đặc điểm chính

- I2C (Inter-Integrated Circuit) là giao thức truyền thông nối tiếp hai dây gồm SDA (dữ liệu) và SCL (đồng hồ).
- Hỗ trợ kết nối nhiều thiết bị trên cùng một bus mà chỉ cần hai dây tín hiệu.
- Tốc độ truyền dữ liệu trung bình (thường 100kHz hoặc 400kHz), phù hợp truyền tải dữ liệu cảm biến.
- Hỗ trợ địa chỉ thiết bị giúp phân biệt và quản lý nhiều cảm biến/màn hình trên cùng một bus.
- Đơn giản, tiết kiệm chân kết nối trên vi điều khiển.

Lý do chọn giao thức I2C

- **Tiết kiệm chân kết nối:** Với chỉ hai dây cho nhiều thiết bị, giúp giảm bớt



số chân sử dụng trên ESP32, thuận tiện cho hệ thống nhiều cảm biến.

- **Dễ dàng mở rộng:** Có thể thêm hoặc thay đổi cảm biến, màn hình mà không cần thêm dây kết nối mới.
- **Đơn giản trong thiết kế phần cứng và phần mềm:** ESP32 hỗ trợ giao tiếp I2C sẵn, thư viện phong phú, dễ lập trình.
- **Tốc độ truyền dữ liệu đủ dùng:** Đáp ứng tốt yêu cầu truyền dữ liệu từ các cảm biến BH1750, SHT31 và màn hình OLED.
- **Khả năng đồng bộ cao:** Tránh xung đột dữ liệu khi nhiều thiết bị chia sẻ cùng bus.

3.2: Nguyên lý hoạt động của các phần tử trong hệ thống

3.2.1: ESP32 (Vi điều khiển trung tâm)

- **Nguyên lý:**
ESP32 là một vi điều khiển tích hợp **WiFi, Bluetooth**, cùng nhiều giao tiếp như **I2C, SPI, UART, PWM, ADC...** giúp điều khiển và giao tiếp với nhiều thiết bị ngoại vi cùng lúc.
Nó có thể đọc dữ liệu từ cảm biến, xử lý, hiển thị, truyền lên nền tảng đám mây (Firebase), và nhận lệnh điều khiển từ xa.
- **Vai trò:**
 - Đọc dữ liệu từ BH1750 và SHT31 thông qua I2C.
 - Hiển thị thông tin lên màn hình OLED.
 - Gửi dữ liệu môi trường lên Firebase Realtime Database.
 - Nhận lệnh từ Firebase để điều khiển Servo SG90.
 - Xử lý logic và phản hồi nhanh theo yêu cầu giám sát.

3.2.2: Cảm biến ánh sáng BH1750

- **Nguyên lý:**
BH1750 sử dụng **photodiode** để đo cường độ ánh sáng. Ánh sáng tác động lên diode tạo ra dòng điện tỉ lệ, được chuyển sang tín hiệu số bằng bộ ADC tích hợp sẵn.
- **Tín hiệu ra:** Giao tiếp I2C, dữ liệu trả về là giá trị lux.
- **Vai trò:** Đo và giám sát cường độ ánh sáng môi trường.

3.2.3:Cảm biến nhiệt độ/độ ẩm SHT31

- **Nguyên lý:**
SHT31 đo độ ẩm bằng **cảm biến điện dung** và đo nhiệt độ bằng **điện trở nhiệt**. Tín hiệu analog được chuyển đổi thành dữ liệu số nhờ bộ vi xử lý bên trong cảm biến.
 - **Tín hiệu ra:** I2C, trả về nhiệt độ (°C) và độ ẩm (%RH).
 - **Vai trò:** Theo dõi nhiệt độ và độ ẩm trong không gian chuồng trại.
-

3.2.4:Màn hình OLED 1.3 inch (I2C)

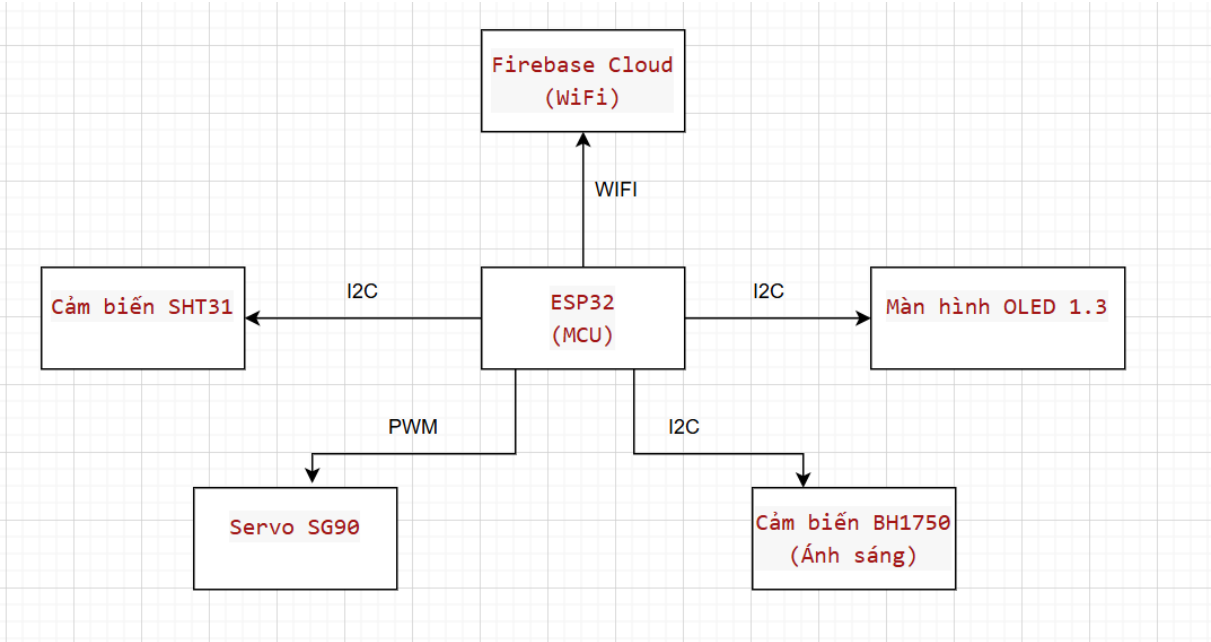
- **Nguyên lý:**
Mỗi điểm ảnh là một **điốt phát sáng hữu cơ (OLED)**, tự phát sáng mà không cần đèn nền. Màn hình nhận dữ liệu từ ESP32 và hiển thị thông qua giao tiếp I2C.
 - **Tín hiệu vào:** Dữ liệu văn bản hoặc hình ảnh từ ESP32.
 - **Vai trò:** Hiển thị trực tiếp các thông số môi trường cho người dùng theo dõi.
-

3.2.5:Servo SG90

- **Nguyên lý:**
SG90 là **servo điều khiển góc quay**, điều khiển bởi xung PWM. Tùy vào độ rộng xung, servo quay đến các vị trí tương ứng từ 0 đến 180 độ. Bên trong gồm động cơ DC nhỏ, bánh răng và biến trở hồi tiếp vị trí.
- **Tín hiệu điều khiển:** PWM từ ESP32 (thường là tần số 50Hz).
- **Vai trò:** Đóng vai trò cơ cấu chấp hành, ví dụ: mở cửa, gạt lỗ thông khí hoặc phản hồi tự động theo điều kiện môi trường.

4. Thiết kế hệ thống:

4.1: Sơ đồ khối hệ thống:



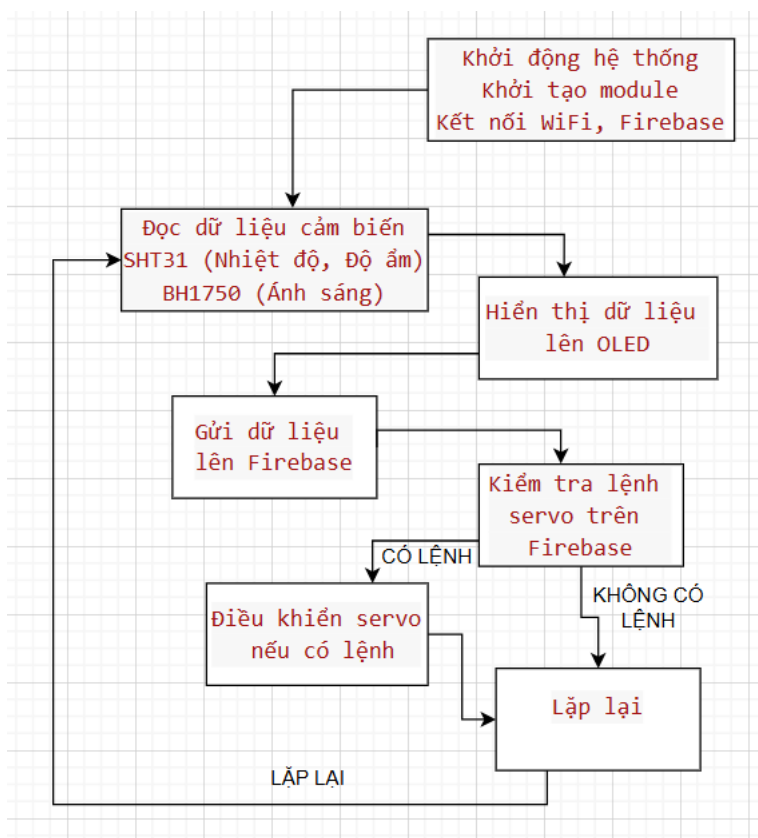
4.2: Sơ đồ nối chân:

Thiết bị	Chân ESP32	Chân thiết bị	Chức năng / Ghi chú
SHT31	GPIO 21 (SDA)	SDA	Dữ liệu I2C
	GPIO 22 (SCL)	SCL	Đồng hồ I2C
	3.3V	VCC	Nguồn cấp 3.3V
	GND	GND	Mass chung
BH1750	GPIO 21 (SDA)	SDA	Dữ liệu I2C (chung bus với SHT31)
	GPIO 22 (SCL)	SCL	Đồng hồ I2C (chung bus)
	3.3V	VCC	Nguồn cấp 3.3V
	GND	GND	Mass chung
OLED 1.3"	GPIO 21 (SDA)	SDA	Dữ liệu I2C (chung bus)
	GPIO 22 (SCL)	SCL	Đồng hồ I2C (chung bus)
	3.3V hoặc 5V	VCC	Nguồn cấp (tùy module OLED)
	GND	GND	Mass chung
Servo SG90	GPIO 25	Signal	Tín hiệu PWM điều khiển servo
	5V	VCC	Nguồn 5V (nguồn ngoài hoặc ESP32)
	GND	GND	Mass chung

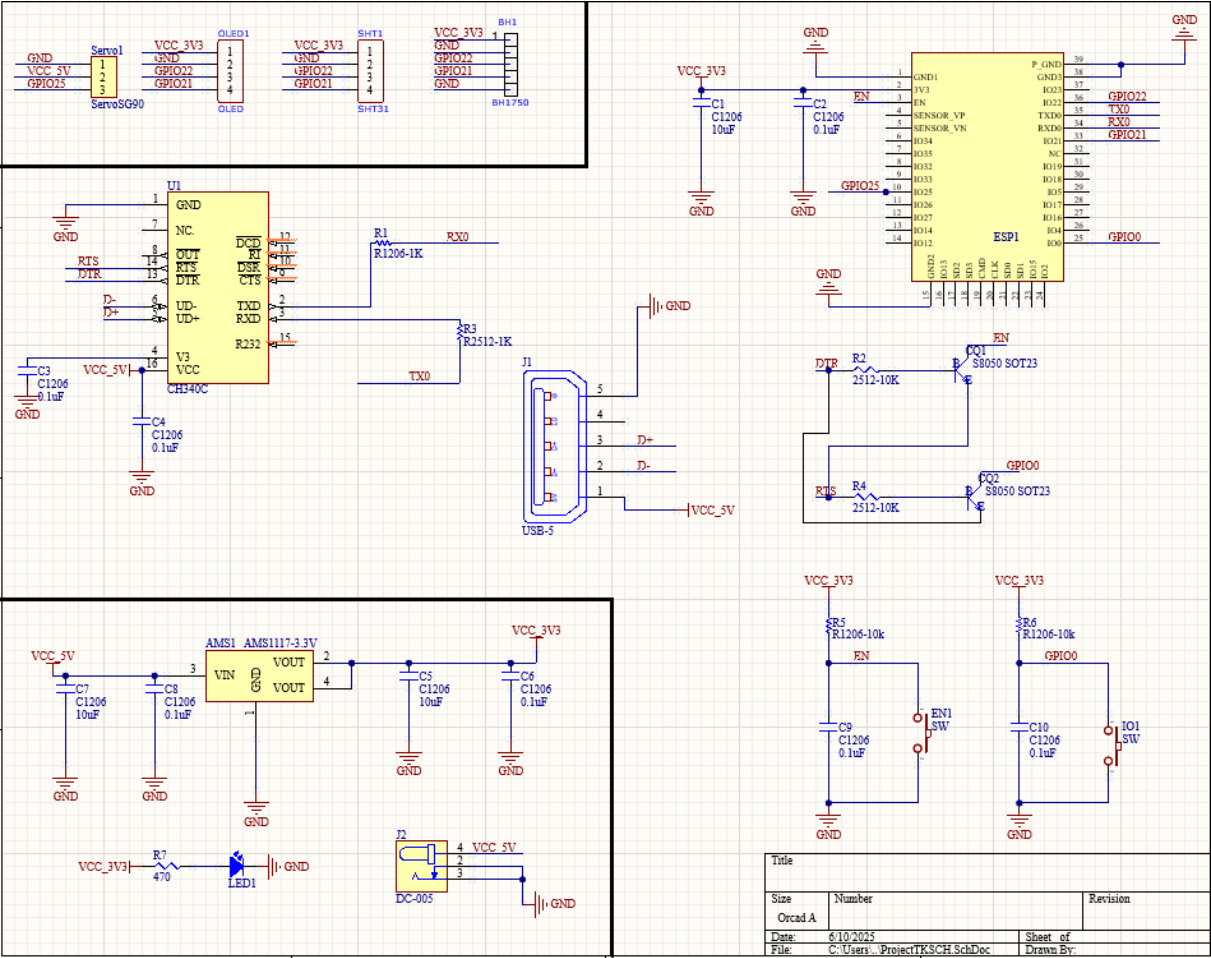
4.3: Giải thích cách các module kết nối với nhau:

1. **ESP32** là bộ vi điều khiển trung tâm, chịu trách nhiệm đọc dữ liệu từ các cảm biến, xử lý và gửi dữ liệu lên Firebase qua WiFi, đồng thời điều khiển servo theo lệnh từ Firebase hoặc chương trình điều khiển.
2. **Cảm biến SHT31 (cảm biến nhiệt độ và độ ẩm)** và **BH1750 (cảm biến ánh sáng)** cùng sử dụng giao tiếp **I2C**.
 - Hai cảm biến này được kết nối chung trên cùng một bus I2C gồm 2 dây SDA (GPIO21) và SCL (GPIO22) của ESP32.
 - Việc dùng chung bus I2C giúp tiết kiệm chân và dây kết nối, đồng thời dễ dàng mở rộng thêm thiết bị I2C khác nếu cần.
3. **Màn hình OLED 1.3"** cũng dùng giao tiếp **I2C** với 2 dây SDA và SCL chung với các cảm biến trên, giúp hiển thị trực tiếp các thông số đo được như nhiệt độ, độ ẩm và cường độ ánh sáng.
 - Nguồn màn hình được cấp 3.3V hoặc 5V tùy module (phải kiểm tra datasheet màn hình để cấp nguồn đúng).
4. **Servo SG90** sử dụng chân GPIO25 làm tín hiệu PWM để điều khiển góc quay của servo.
 - Servo cần nguồn 5V riêng vì ESP32 không cung cấp đủ dòng, mass của servo và ESP32 được nối chung để đảm bảo cùng mức điện áp.
5. **Nguồn cấp:**
 - ESP32 và các cảm biến dùng nguồn 3.3V ổn định.
 - Servo dùng nguồn 5V riêng biệt nhưng mass chung với ESP32 để tránh nhiễu và mất ổn định điện áp.
6. **Kết nối WiFi và Firebase:**
 - ESP32 kết nối mạng WiFi để gửi dữ liệu đo được lên **Firestore Realtime Database**.
 - Firebase cũng có thể gửi lệnh điều khiển servo ngược lại ESP32 qua mạng Internet.

4.4: Lưu đồ hoạt động hoặc biểu đồ trạng thái:



4.5: Sơ đồ nguyên lý của mạch:



Dự án sử dụng các linh kiện:

Tên linh kiện:	Chức năng:	Số Lượng:
AMS1117-3.3V	Hạ áp từ 5V xuống 3.3V cấp cho ESP32	1
BH1750	Cảm biến đo cường độ ánh sáng, giao tiếp I2C	1
C1206(10uF và 0.1uF)	Tụ gốm (lọc nhiễu, ổn định nguồn)	10
SW	Nút nhấn (reset hoặc boot cho ESP32)	2
ESP32	Vi điều khiển chính, xử lý và điều khiển hệ thống	1
USB-5	Cổng cấp nguồn 5V adapter(5V 3A)	1
DC-005	Jack cắm nguồn DC từ adapter ngoài	1
LED	LED báo nguồn	1

OLED	Màn hình hiển thị thời gian thông số cảm biến, dùng I2C	1
S8050 SOT23	Transistor NPN, dùng để auto-reset và boot nạp ESP	2
R1206-1K	Điện trở 1k Ω chân dán (SMD)	1
R2512-10K	Điện trở 10k Ω chân dán(SMD)	2
R2512-1K	Điện trở 1k Ω chân dán(SMD)	1
R1206-10k	Điện trở 10k Ω chân dán(SMD)	2
R470	Điện trở 470 Ω cho LED	1
ServoSG90	Động cơ servo điều khiển thiết bị cơ khí(cửa,...)	1
SHT31	Cảm biến đo nhiệt độ và độ ẩm, giao tiếp I2C	1
CH340C	IC chuyển USB sang UART, dùng để nạp code và giao tiếp với ESP	1
Adapter 5V 3A	Adapter cấp điện cho thiết bị	1

Dự án được chia làm 3 khối:

1. Khối nguồn – Cung cấp và ổn định điện áp cho toàn hệ thống

Chức năng: Hạ áp, lọc nhiễu, chia nhánh nguồn cho ESP32 và thiết bị ngoại vi

Linh kiện gồm:

- **DC-005** – Jack cắm nguồn từ adapter 5V
- **AMS1117-3.3V** – Hạ áp 5V xuống 3.3V
- **C1206 (10uF, 0.1uF)** – Tụ gốm lọc nhiễu đầu vào/ra của AMS1117
- **R2512-10K, R2512-1K, R1206-10K, R1206-1K, 470 Ω** – Điện trở lọc hoặc chia áp (nếu dùng trong mạch nguồn)
- **USB-5** – Cổng cấp nguồn từ USB (tùy chọn)

2. Khối xử lý – Điều khiển, xử lý tín hiệu và giao tiếp

Chức năng: Đọc dữ liệu cảm biến, điều khiển ngoại vi, giao tiếp WiFi, xử lý dữ liệu phần CLOUD

Linh kiện gồm:

- **ESP32** – Vi điều khiển chính
 - **CH340C** – Nạp code, giao tiếp UART
 - **S8050 (Q1, Q2)** – Điều khiển auto-reset và bootloader
 - **SW** – Nút nhấn reset/boot
 - **LED** – Báo trạng thái hoạt động hoặc debug
-

3. Khối ngoại vi – Cảm biến và thiết bị được điều khiển

Chức năng: Cung cấp thông tin đầu vào và thực hiện điều khiển thực tế

Linh kiện gồm:

- **BH1750** – Cảm biến đo ánh sáng (I2C)
- **SHT31** – Cảm biến nhiệt độ, độ ẩm (I2C)
- **OLED** – Màn hình hiển thị dữ liệu (I2C)
- **Servo SG90** – Thiết bị cơ điện, điều khiển chuyển động (PWM)

5. Hệ thống IOT:

Hệ thống IoT sử dụng Firebase kết hợp với giao diện web giúp thu thập và hiển thị dữ liệu cảm biến thời gian thực. Firebase đảm bảo đồng bộ nhanh, bảo mật cao, trong khi web cho phép người dùng dễ dàng quan sát và điều khiển thiết bị từ xa với **giao diện tùy biến với độ linh hoạt cực kỳ cao**.

Hệ thống IoT được sử dụng trong dự án bao gồm 3 thành phần chính:

1. **Thiết bị cảm biến (ESP32 + cảm biến SHT31, BH1750, SG90, OLED 1.3")**
 - Thu thập dữ liệu môi trường (nhiệt độ, độ ẩm, ánh sáng) và trạng thái servo
 - Gửi dữ liệu lên **Firestore Realtime Database** qua WiFi
2. **Nền tảng đám mây Firebase**
 - Lưu trữ và đồng bộ dữ liệu cảm biến trong thời gian thực
 - Quản lý dữ liệu, bảo mật và phân quyền truy cập
 - Nhận và truyền lệnh điều khiển servo từ giao diện người dùng
3. **Giao diện web (HTML + CSS + JavaScript)**
 - Kết nối trực tiếp với Firebase để hiển thị dữ liệu realtime lên web



- Cung cấp các nút điều khiển thiết bị (bật/tắt servo hoặc điều chỉnh góc quay)
- Thiết kế đơn giản, dễ sử dụng, truy cập được trên mọi thiết bị có trình duyệt

Các nền tảng IoT phổ biến khác

- **Thingspeak:** Thuần tập trung vào thu thập và biểu diễn dữ liệu, phù hợp với dự án nhỏ, không có nhiều chức năng điều khiển. Hạn chế về tốc độ cập nhật và tùy biến giao diện.
- **ThingsBoard:** Mã nguồn mở, hỗ trợ đa dạng giao thức, tùy biến cao, phù hợp doanh nghiệp lớn nhưng đòi hỏi kỹ thuật vận hành và cài đặt khá phức tạp.
- **Blynk:** Tập trung vào điều khiển thiết bị qua app di động, giao diện đẹp, nhưng hạn chế về khả năng tùy biến backend và phần mềm web.
- **Web Local ESP32:** Không cần internet, thiết kế cho mạng nội bộ, rất phù hợp demo hoặc hệ thống nhỏ, nhưng không hỗ trợ điều khiển từ xa hoặc quản lý quy mô lớn.

Vì sao chọn Firebase + Web thay vì các nền tảng khác?

Ưu điểm

- **Realtime Database cực nhanh và ổn định** giúp cập nhật dữ liệu cảm biến và trạng thái thiết bị gần như ngay lập tức.
- **Dễ dàng tích hợp với ESP32** nhờ các thư viện hỗ trợ sẵn, giảm thiểu thời gian phát triển firmware.
- **Bảo mật và phân quyền chi tiết** giúp kiểm soát ai được truy cập, sửa dữ liệu.
- **Giao diện web tùy biến linh hoạt:** Tự thiết kế HTML/CSS giúp tùy chỉnh giao diện theo nhu cầu, không bị gò bó bởi mẫu có sẵn như Blynk hay Thingspeak.
- **Phát triển mở rộng dễ dàng:** Firebase thuộc hệ sinh thái Google, có thể kết hợp nhiều dịch vụ khác như Authentication, Cloud Functions, Hosting, giúp hệ thống dễ dàng phát triển thêm tính năng.
- **Không cần quản lý server:** Giảm thiểu chi phí và công sức vận hành hệ thống backend.

Nhược điểm

- **Cần kiến thức lập trình web và Firebase** để xây dựng giao diện và backend.

- Phụ thuộc internet và dịch vụ đám mây **Firebase**, không phù hợp các hệ thống cần hoạt động offline hoặc mạng nội bộ.
- Chi phí tăng lên khi mở rộng lớn, cần cân nhắc quản lý tài nguyên.

6. Thiết kế phần mềm:

6.1: Môi trường lập trình sử dụng (PlatformIO):

Môi trường lập trình sử dụng

- **PlatformIO:**
PlatformIO là môi trường phát triển tích hợp (IDE) mạnh mẽ, đa nền tảng, hỗ trợ lập trình cho ESP32 và nhiều vi điều khiển khác. PlatformIO cung cấp công cụ quản lý thư viện, xây dựng dự án tự động, hỗ trợ debug và nạp chương trình dễ dàng, giúp tối ưu quy trình phát triển phần mềm.

6.2: Thư viện sử dụng:

```
#include <Wire.h>           // Giao tiếp I2C cho cảm biến và OLED
#include <U8g2lib.h>         // Thư viện điều khiển màn hình OLED 1.3 inch
#include <ESP32Servo.h>      // Thư viện điều khiển servo SG90 trên ESP32
#include <Adafruit_SHT31.h>  // Thư viện cảm biến nhiệt độ và độ ẩm SHT31
#include <BH1750.h>          // Thư viện cảm biến ánh sáng BH1750
#include <WiFi.h>            // Thư viện kết nối WiFi cho ESP32
#include <time.h>             // Thư viện xử lý thời gian thực
#include <FirebaseESP32.h>   // Thư viện kết nối và làm việc với Firebase
```

- **Wire.h:** Dùng để giao tiếp I2C giữa ESP32 và các cảm biến, màn hình OLED.
- **U8g2lib.h:** Hỗ trợ hiển thị dữ liệu trên màn hình OLED 1.3 inch với độ phân giải cao.
- **ESP32Servo.h:** Điều khiển servo SG90 qua chân IO25 với độ chính xác và ổn định.
- **Adafruit_SHT31.h:** Đọc dữ liệu nhiệt độ và độ ẩm từ cảm biến SHT31.
- **BH1750.h:** Đọc dữ liệu cường độ ánh sáng từ cảm biến BH1750.
- **WiFi.h:** Kết nối ESP32 vào mạng WiFi để giao tiếp với Firebase.
- **time.h:** Đồng bộ và xử lý thời gian thực cho hệ thống (nếu cần).
- **FirebaseESP32.h:** Thư viện chính để giao tiếp, đọc và ghi dữ liệu với Firebase Realtime Database.

6.3: Các hàm chính (đọc cảm biến, hiển thị OLED, điều khiển servo, gửi Firebase):

6.3.1: Hàm setup() – Khởi tạo hệ thống

```
void setup() {  
  Serial.begin(115200);  
  Wire.begin(21, 22);           // I2C cho cảm biến và OLED  
  u8g2.begin();                 // Khởi động OLED  
  myServo.attach(13);           // Gán servo vào chân D13  
  
  WiFi.begin(ssid, password);   // Kết nối WiFi  
  ...  
  setupTime();                  // Đồng bộ thời gian NTP  
  
  sht31.begin(0x44);            // Khởi tạo SHT31  
  lightMeter.begin(...);         // Khởi tạo BH1750  
  
  config.database_url = FIREBASE_HOST; // Cấu hình Firebase  
  config.signer.tokens.legacy_token = FIREBASE_AUTH;  
  Firebase.begin(&config, &auth);  
}
```

codesnap.dev

Chức năng:

- Thiết lập giao tiếp I2C, kết nối WiFi, đồng bộ thời gian, khởi tạo cảm biến và Firebase.

6.3.2:Hàm setupTime() – Đồng bộ thời gian thực từ NTP:

```
void setupTime() {  
    configTime(7 * 3600, 0, "pool.ntp.org", "time.nist.gov");  
    while (time(nullptr) < 100000) delay(500);  
}
```

codesnap.dev

Chức năng: Lấy giờ thực từ Internet để hiển thị chính xác thời gian lên OLED.

6.3.3: Điều khiển Servo qua Firebase:

```
if (Firebase.getInt(fbdo, "/servoAngle")) {  
    int angle = fbdo.intData();  
    myServo.write(angle);           // Điều khiển Servo theo Firebase  
}
```

codesnap.dev

Chức năng: Nhận lệnh điều khiển từ Firebase (node /servoAngle), điều chỉnh góc servo theo yêu cầu.

6.3.4: Đọc cảm biến và gửi lên Firebase:

```
currentLight = lightMeter.readLightLevel();
Firebase.setFloat(fbdo, "/light", currentLight);

currentTemp = sht31.readTemperature();
Firebase.setFloat(fbdo, "/temp", currentTemp);

currentHum = sht31.readHumidity();
Firebase.setFloat(fbdo, "/humidity", currentHum);
```

Chức năng: Đọc giá trị ánh sáng, nhiệt độ và độ ẩm rồi gửi lên Firebase Realtime Database theo nhánh tương ứng.

6.3.5: Hiển thị OLED:

```
u8g2.setCursor(54, 10);
u8g2.print("Light: "); u8g2.print(currentLight, 0);

u8g2.setCursor(54, 25);
u8g2.print("Temp: "); u8g2.print(currentTemp, 1); u8g2.print(" C");

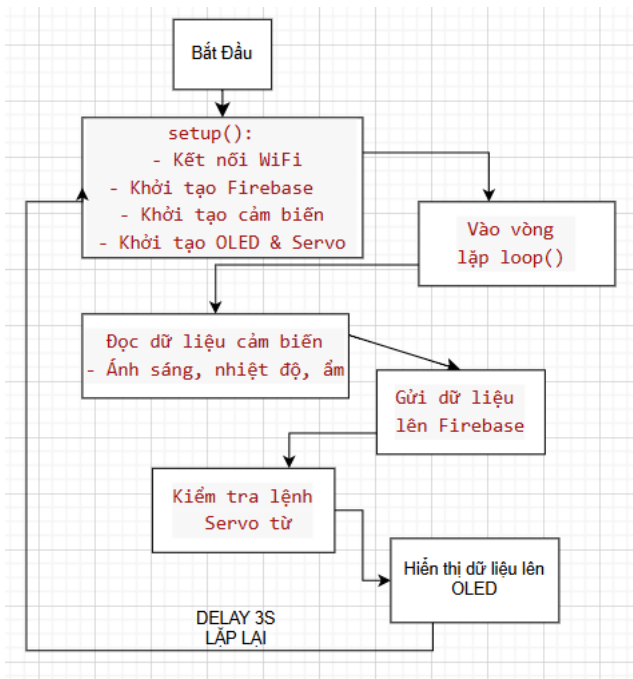
u8g2.setCursor(54, 40);
u8g2.print("Hum: "); u8g2.print(currentHum, 1); u8g2.print(" %");

u8g2.setCursor(54, 55);
u8g2.print("Servo: "); u8g2.print(servoAngleDisplay);
```

codesnap.dev

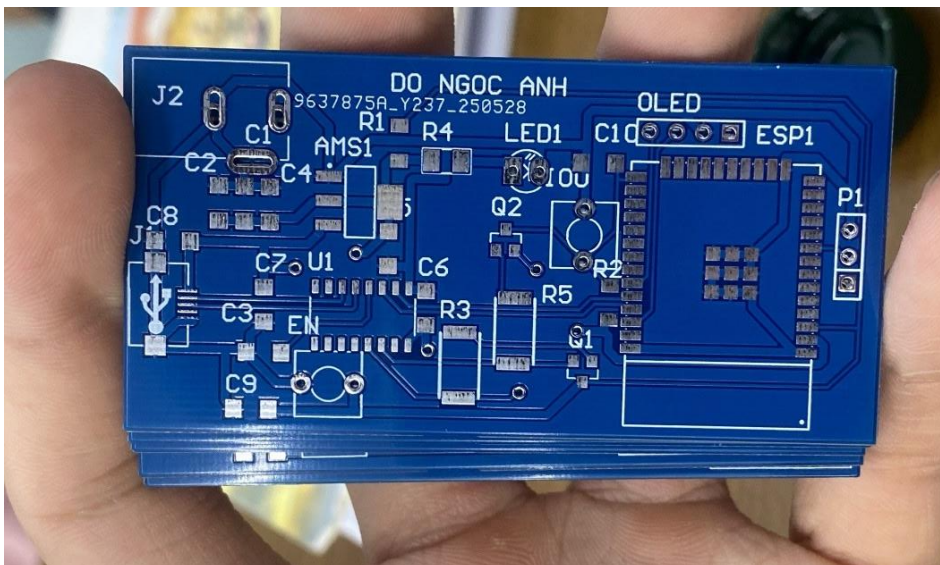
Chức năng: Hiển thị giá trị cảm biến và trạng thái servo trên màn hình OLED SH1106.

6.4: Giải thích luồng chương trình:

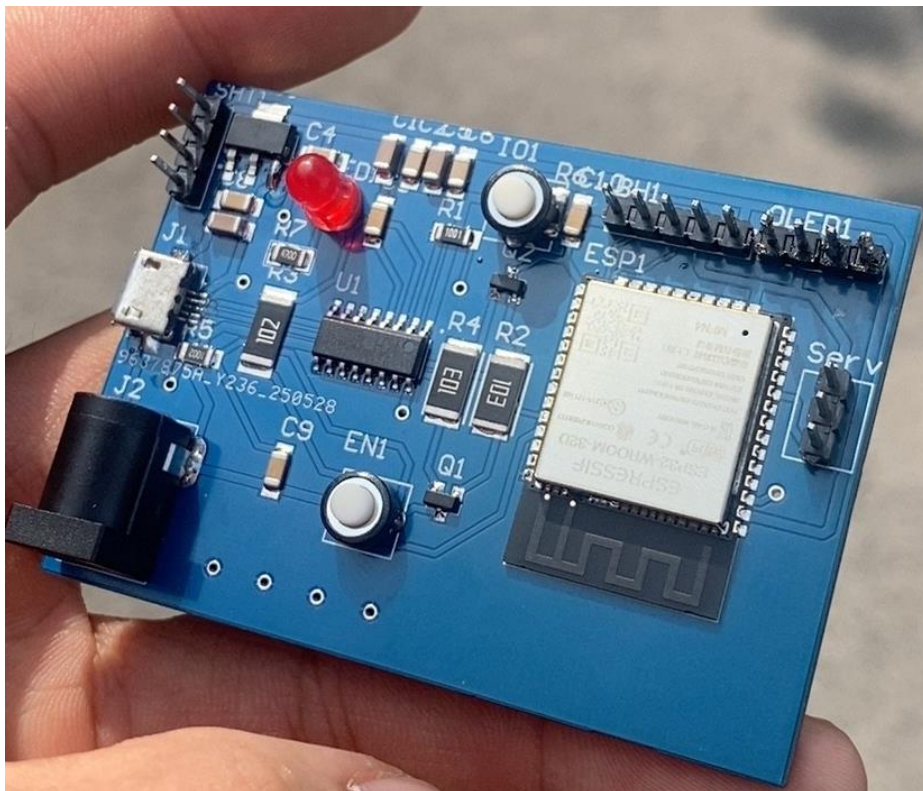


7.Hình Ảnh Thực Tế:

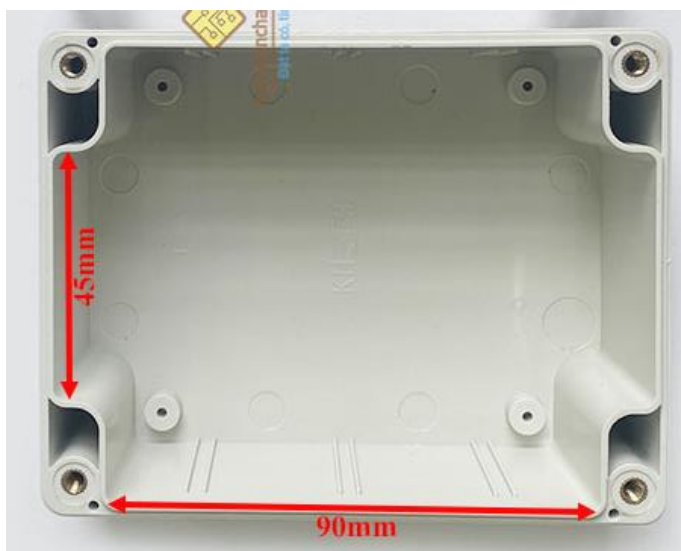
7.1:Mạch chưa hàn:



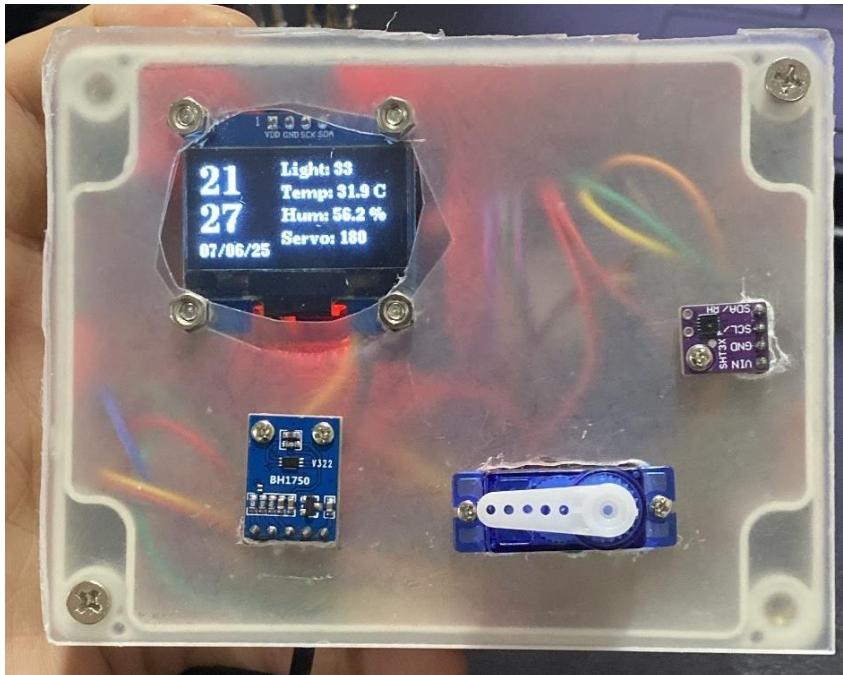
7.2:Mạch đã hàn:



7.3:Hộp Dựng:



7.4:Sản Phẩm:



Sản phẩm cuối có nhiều thiếu sót và những đường nét em còn chưa làm đẹp,mong thầy bỏ qua cho em ạ.